

PROJECT REPORT

Database System for Event Registration & Venue Scheduling

(Prototype: EVENTRA – Venue OS)



Programme: B.Tech CSE

Class / Semester: CSE Tigers | Semester 3

Course: Database Management Systems (DBMS) – Project Based Learning

Submitted By:

Devesh Wattamwar (25WU0101030)

Sarthak Borse (25WU0101025)

Dhruv Porwal (25WU0101031)

Under Supervision of:

Dr. Kiran Mayee Adavala

University: Woxsen University

Academic Year: 2026 – 27

Submission Date: 7 October 2026

Table of Contents

1. Cover Page	1
2. Abstract	3
3. Introduction and Problem Statement	3
4. Objectives and Scope	4
5. Software and Hardware Requirements	5
6. ER Diagram	5
7. Relational Schema and Normalization	6
8. Data Dictionary	8
9. SQL Commands Used (DDL, DML) with Sample Outputs	11
10. Queries with Outputs	16
11. UI Design and Screenshots	19
12. Implementation Details	23
13. Testing	24
14. Conclusion and Future Enhancements	26
15. References	26
16. Contribution of Each Member	27
17. Appendix: GitHub Repository Link	27

2. Abstract

Educational institutions run many seminars, workshops and cultural events every semester, but most of them still manage registrations, venue bookings, attendance and certificates on paper or in separate spreadsheets. This causes duplicate entries, venue clashes, overcrowded halls and certificates that cannot be justified with proper records.

This project designs and implements a relational database, **Event Registration and Venue Scheduling System**, that keeps all of this data in one place. The design has **12 entities** (Category, Coordinator, Venue, Speaker, Participant, Event, Session, Registration, Payment, Attendance, Certificate and Feedback) and is normalized up to **Third Normal Form**. Primary keys, foreign keys, UNIQUE, NOT NULL and CHECK constraints, together with three triggers, enforce the business rules directly in the database: no venue overlap, no registration beyond capacity, no duplicate registration, certificates only after 75% attendance, and no negative payments.

The system supports reports using joins, GROUP BY, HAVING, subqueries and views, and a web-based prototype (**EVENTRA – Venue OS**), built with Next.js, TypeScript and Drizzle ORM, with separate views for Administrator, Organizer and Participant for inserting, viewing and deleting records. Test cases show that valid data is accepted and invalid data is rejected by the database itself.

Keywords: DBMS, ER model, normalization, 3NF, MySQL, constraints, triggers, event management.

3. Introduction and Problem Statement

3.1 Introduction

- Colleges organise seminars, workshops, hackathons and cultural fests throughout the year.
- Each event needs the same basic things: a venue, a time slot, speakers, participant registration, attendance, payment collection, certificates and feedback.
- When these are handled manually, data is scattered across notebooks, spreadsheets and messages, and nobody has a single correct view.
- A database management system gives one shared, validated and queryable store for all of this information.

3.2 Problem Statement

To design and build a relational database system that manages events, sessions, venues, participant registrations, attendance, payments, certificates and feedback, while preventing venue conflicts, capacity overruns, duplicate registrations and invalid payments.

3.3 Problems in the Manual System

Current practice	Consequence
Registrations on paper and spreadsheets	Lost entries, misspelled names, no single source of truth
Venues booked on noticeboards or shared documents	Double-bookings and scheduling clashes
Capacity tracked from the coordinator's memory	Overcrowded halls and fire-safety risk
Attendance marked on a clipboard	No audit trail, no basis for certificates
Cash and receipts reconciled by hand	Discrepancies and lost revenue
Feedback collected inconsistently	No data to improve future events

3.4 Proposed Solution

- One central database with 12 normalized tables acts as the single source of truth.
- Rules are enforced by the database (constraints and triggers), not by trust.
- Reports are produced with SQL joins, aggregates and views.

4. Objectives and Scope

4.1 Objectives

1. Design a reliable relational schema modeled on real event workflows, normalized to 3NF.
2. Manage events and their sessions, including speakers, venues and timings.
3. Prevent venue conflicts: no double-bookings and no capacity overruns.
4. Manage registrations: one participant per event, no duplicates, with a record of status and date.
5. Track per-session attendance and per-registration payments.
6. Automate certificate eligibility: issue only when the attendance threshold (75%) is met.
7. Generate reports using joins, GROUP BY, HAVING and views, and maintain integrity with PK/FK rules.

4.2 Scope

In scope	Out of scope (future work)
Events, categories and sessions Venues and capacity checks Participant registration Payments (recorded in the database) and attendance Certificates and feedback Reports and views Web UI prototype (EVENTRA) for insert, view and delete	Online payment gateway Email and SMS notifications QR-code based attendance Mobile application Cloud deployment

4.3 Intended Users

User	Role in the system	Main duties
Administrator	Owns configuration	Create and manage events; maintain venues and capacity; maintain speakers and categories; issue organisation-wide reports
Participant	End-user	Browse and search events; register and pay; view own registrations; submit feedback; download certificate
Coordinator (Organizer in the UI)	Operator at the events desk	Schedule sessions and venues; assign speakers; record attendance; mark certificate eligibility; run session reports

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Details
Operating system	Windows 10/11 or Ubuntu 22.04+
Database design and SQL scripts	MySQL 8.0 (Community Edition) with MySQL Workbench / command-line client; scripts also tested on MariaDB 10.11
Database used by the prototype	PostgreSQL, accessed through node-postgres (pg)
Runtime	Node.js (LTS) with npm
Web framework	Next.js 16 with React 19 and TypeScript 5
Styling	Tailwind CSS 4 (through PostCSS)
ORM and migrations	Drizzle ORM and drizzle-kit
Other libraries	qrcode (QR tickets), dotenv (environment variables), ESLint
Browser	Latest Google Chrome or Microsoft Edge
Editor / IDE	Visual Studio Code
Version control	Git and GitHub

5.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / Ryzen 5 or better
RAM	4 GB	8 GB
Storage	2 GB free space	10 GB free space (SSD)
Display	1366 x 768	1920 x 1080
Network	Not required (local database)	Required only for GitHub access

6. ER Diagram

The ER model has 12 entities and 12 relationships. EVENT and REGISTRATION are the two hubs of the design.

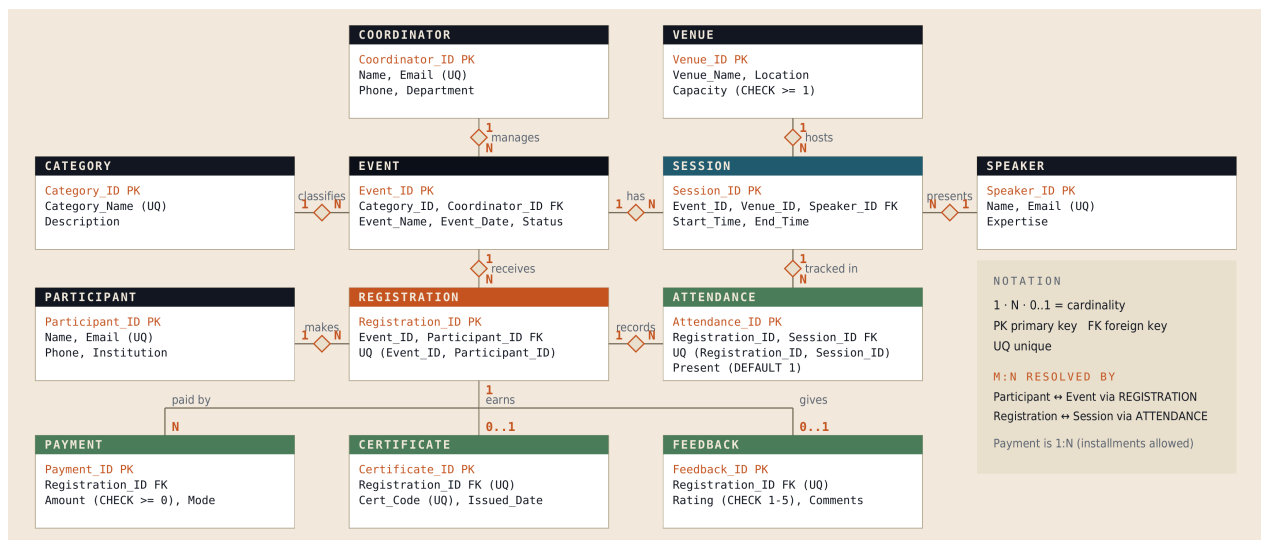


Figure 6.1 – Entity-Relationship diagram (PK = primary key, FK = foreign key, UQ = unique, 1 / N / 0..1 = cardinality)

6.1 Entities, Attributes and Keys

Entity	Primary key	Other attributes
Category	Category_ID	Category_Name (UQ), Description
Coordinator	Coordinator_ID	Name, Email (UQ), Phone, Department
Venue	Venue_ID	Venue_Name, Location, Capacity (CHECK >= 1)
Speaker	Speaker_ID	Name, Email (UQ), Expertise, Bio, Contact_Phone
Participant	Participant_ID	Name, Email (UQ), Phone, Institution
Event	Event_ID	Event_Name, Event_Date, Description, Status; FK: Category_ID, Coordinator_ID
Session	Session_ID	Title, Start_Time, End_Time; FK: Event_ID, Venue_ID, Speaker_ID
Registration	Registration_ID	Registration_Date, Status; FK: Event_ID, Participant_ID; UQ(Event_ID, Participant_ID)
Payment	Payment_ID	Amount (CHECK >= 0), Mode, Payment_Date, Status; FK: Registration_ID
Attendance	Attendance_ID	Marked_At, Present; FK: Registration_ID, Session_ID; UQ(Registration_ID, Session_ID)
Certificate	Certificate_ID	Issued_Date, Cert_Code (UQ); FK: Registration_ID (UQ)
Feedback	Feedback_ID	Rating (CHECK 1–5), Comments, Submitted_At; FK: Registration_ID (UQ)

6.2 Relationships, Cardinality and Participation

Relationship	Entities	Cardinality	Participation
classifies	Category – Event	1 : N	Category partial, Event total
manages	Coordinator – Event	1 : N	Coordinator partial, Event total
has	Event – Session	1 : N	Event partial, Session total
hosts	Venue – Session	1 : N	Venue partial, Session total
presents	Speaker – Session	1 : N	Speaker partial, Session partial (speaker optional)
receives	Event – Registration	1 : N	Event partial, Registration total
makes	Participant – Registration	1 : N	Participant partial, Registration total
records	Registration – Attendance	1 : N	Registration partial, Attendance total
tracked in	Session – Attendance	1 : N	Session partial, Attendance total
paid by	Registration – Payment	1 : N	Registration partial, Payment total
earns	Registration – Certificate	1 : 0..1	Registration partial, Certificate total
gives	Registration – Feedback	1 : 0..1	Registration partial, Feedback total

Many-to-many relationships

- **Participant** ↔ **Event** is M:N, resolved by the REGISTRATION table (with UNIQUE(Event_ID, Participant_ID)).
- **Registration** ↔ **Session** is M:N, resolved by the ATTENDANCE table (with UNIQUE(Registration_ID, Session_ID)).

7. Relational Schema and Normalization

7.1 Relational Schema

Notation: **PK** is underlined and bold, *FK* is in italics.

Relation	Attributes
Category	<u>Category_ID</u> , Category_Name, Description
Coordinator	<u>Coordinator_ID</u> , Name, Email, Phone, Department
Venue	<u>Venue_ID</u> , Venue_Name, Location, Capacity
Speaker	<u>Speaker_ID</u> , Name, Email, Expertise, Bio, Contact_Phone
Participant	<u>Participant_ID</u> , Name, Email, Phone, Institution
Event	<u>Event_ID</u> , Event_Name, Category_ID, Coordinator_ID, Event_Date, Description, Status
Session	<u>Session_ID</u> , Event_ID, Venue_ID, Speaker_ID, Title, Start_Time, End_Time
Registration	<u>Registration_ID</u> , Event_ID, Participant_ID, Registration_Date, Status [UNIQUE (Event_ID, Participant_ID)]
Payment	<u>Payment_ID</u> , Registration_ID, Amount, Mode, Payment_Date, Status
Attendance	<u>Attendance_ID</u> , Registration_ID, Session_ID, Marked_At, Present [UNIQUE (Registration_ID, Session_ID)]
Certificate	<u>Certificate_ID</u> , Registration_ID, Issued_Date, Cert_Code
Feedback	<u>Feedback_ID</u> , Registration_ID, Rating, Comments, Submitted_At

7.2 Functional Dependencies

No.	Functional dependency
FD1	Participant_ID → Name, Email, Phone, Institution
FD2	Venue_ID → Venue_Name, Location, Capacity
FD3	Event_ID → Event_Name, Event_Date, Category_ID, Coordinator_ID, Status
FD4	Session_ID → Event_ID, Venue_ID, Speaker_ID, Start_Time, End_Time
FD5	Registration_ID → Event_ID, Participant_ID, Registration_Date, Status
FD6	Category_ID → Category_Name, Description

7.3 Normalization (UNF to 3NF)

A single registration sheet that stores participant, event, category and venue data together is used as the starting point.

Unnormalized form (UNF)

```
REG_SHEET(Participant_ID, P_Name, P_Email,
          { Event_ID, Event_Name, Category_Name, Venue_Name, Capacity })
```

Problem: repeating group in one cell; venue capacity repeated, which leads to update, insert and delete anomalies.

First Normal Form (1NF) – remove repeating groups

```
REG_1NF(Participant_ID, Event_ID, P_Name, P_Email, Event_Name,
        Category_ID, Category_Name, Venue_ID, Venue_Name, Capacity)
Key = (Participant_ID, Event_ID)
```

All values are atomic. **Still wrong:** Participant_ID → P_Name, P_Email and Event_ID → Event_Name ... depend on only part of the key (partial dependency).

Second Normal Form (2NF) – remove partial dependencies

```
PARTICIPANT(Participant_ID, Name, Email)
REGISTRATION(Participant_ID, Event_ID, Reg_Date)
EVENT_INFO(Event_ID, Event_Name, Category_ID, Category_Name,
           Venue_ID, Venue_Name, Capacity)
```

Every non-key attribute now depends on the whole key. **Still wrong:** Event_ID → Category_ID → Category_Name and Venue_ID → Venue_Name, Capacity are transitive dependencies.

Third Normal Form (3NF) – remove transitive dependencies

```

PARTICIPANT(Participant_ID, Name, Email, Phone, Institution)
EVENT(Event_ID, Event_Name, Category_ID, Coordinator_ID, Event_Date, ...)
CATEGORY(Category_ID, Category_Name, Description)
VENUE(Venue_ID, Venue_Name, Location, Capacity)
SESSION(Session_ID, Event_ID, Venue_ID, Speaker_ID, Start_Time, End_Time)
REGISTRATION(Registration_ID, Event_ID, Participant_ID, ...)

```

Result

- Every non-key attribute depends on the key, the whole key and nothing but the key.
- Each fact is stored once: a venue capacity is changed in one row of VENUE only.
- In all 12 final relations the left side of every FD is a candidate key, so the schema is also in BCNF.
- Venue is linked to SESSION (not EVENT) because one event can use different venues for different sessions.

8. Data Dictionary

Table structure, data types and constraints, as created in the database (extracted from the server catalogue).

8.1 Category

Column	Data type	Constraints	Description
Category_ID	INT	PK, AUTO_INCREMENT	Unique id of the category
Category_Name	VARCHAR(60)	NOT NULL, UNIQUE	Category name (Seminar, Workshop ...)
Description	VARCHAR(255)		Short description

8.2 Coordinator

Column	Data type	Constraints	Description
Coordinator_ID	INT	PK, AUTO_INCREMENT	Unique id of the coordinator
Name	VARCHAR(80)	NOT NULL	Full name
Email	VARCHAR(100)	NOT NULL, UNIQUE	Official email
Phone	VARCHAR(15)		Contact number
Department	VARCHAR(60)		Department of the coordinator

8.3 Venue

Column	Data type	Constraints	Description
Venue_ID	INT	PK, AUTO_INCREMENT	Unique id of the venue
Venue_Name	VARCHAR(120)	NOT NULL	Hall / room / lab name
Location	VARCHAR(200)		Building and floor
Capacity	INT	NOT NULL, CHECK (Capacity >= 1)	Maximum seats

8.4 Speaker

Column	Data type	Constraints	Description
Speaker_ID	INT	PK, AUTO_INCREMENT	Unique id of the speaker
Name	VARCHAR(80)	NOT NULL	Full name
Email	VARCHAR(100)	NOT NULL, UNIQUE	Email address
Expertise	VARCHAR(100)		Area of expertise
Bio	VARCHAR(255)		Short biography
Contact_Phone	VARCHAR(15)		Contact number

8.5 Participant

Column	Data type	Constraints	Description
Participant_ID	INT	PK, AUTO_INCREMENT	Unique id of the participant
Name	VARCHAR(80)	NOT NULL	Full name
Email	VARCHAR(100)	NOT NULL, UNIQUE	Email address
Phone	VARCHAR(15)		Contact number
Institution	VARCHAR(100)		College / institution

8.6 Event

Column	Data type	Constraints	Description
Event_ID	INT	PK, AUTO_INCREMENT	Unique id of the event
Event_Name	VARCHAR(120)	NOT NULL	Name of the event
Category_ID	INT	NOT NULL, FK → Category(Category_ID)	Category of the event
Coordinator_ID	INT	NOT NULL, FK → Coordinator(Coordinator_ID)	Coordinator in charge
Event_Date	DATE	NOT NULL	Date of the event
Description	VARCHAR(255)		Short description
Status	VARCHAR(15)	NOT NULL, DEFAULT 'planned', CHECK (Status in ('planned','open','completed','cancelled'))	planned / open / completed / cancelled

8.7 Session

Column	Data type	Constraints	Description
Session_ID	INT	PK, AUTO_INCREMENT	Unique id of the session
Event_ID	INT	NOT NULL, FK → Event(Event_ID)	Event the session belongs to
Venue_ID	INT	NOT NULL, FK → Venue(Venue_ID)	Venue where it is held
Speaker_ID	INT	FK → Speaker(Speaker_ID)	Speaker (optional)
Title	VARCHAR(120)		Session title
Start_Time	DATETIME	NOT NULL, CHECK (End_Time > Start_Time)	Start date and time
End_Time	DATETIME	NOT NULL, CHECK (End_Time > Start_Time)	End date and time

8.8 Registration

Column	Data type	Constraints	Description
Registration_ID	INT	PK, AUTO_INCREMENT	Unique id of the registration
Event_ID	INT	NOT NULL, FK → Event(Event_ID)	Event registered for
Participant_ID	INT	NOT NULL, FK → Participant(Participant_ID)	Participant who registered
Registration_Date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time of registration
Status	VARCHAR(15)	NOT NULL, DEFAULT 'confirmed', CHECK (Status in ('confirmed','cancelled','waitlisted'))	confirmed / cancelled / waitlisted

Composite UNIQUE: (Event_ID,Participant_ID)

8.9 Payment

Column	Data type	Constraints	Description
Payment_ID	INT	PK, AUTO_INCREMENT	Unique id of the payment
Registration_ID	INT	NOT NULL, FK → Registration(Registration_ID)	Registration being paid for
Amount	DECIMAL(10,2)	NOT NULL, CHECK (Amount >= 0)	Amount paid (0 for free events)
Mode	VARCHAR(20)		Cash / UPI / Card / Free
Payment_Date	DATE		Date of payment
Status	VARCHAR(15)	NOT NULL, DEFAULT 'paid', CHECK (Status in ('paid','pending','refunded'))	paid / pending / refunded

8.10 Attendance

Column	Data type	Constraints	Description
Attendance_ID	INT	PK, AUTO_INCREMENT	Unique id of the record
Registration_ID	INT	NOT NULL, FK → Registration(Registration_ID)	Registration being marked
Session_ID	INT	NOT NULL, FK → Session(Session_ID)	Session attended
Marked_At	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of marking
Present	TINYINT(1)	NOT NULL, DEFAULT 1	1 = present, 0 = absent

Composite UNIQUE: (Registration_ID,Session_ID)

8.11 Certificate

Column	Data type	Constraints	Description
Certificate_ID	INT	PK, AUTO_INCREMENT	Unique id of the certificate
Registration_ID	INT	NOT NULL, UNIQUE, FK → Registration(Registration_ID)	Registration it was issued for
Issued_Date	DATE	NOT NULL	Date of issue
Cert_Code	VARCHAR(30)	NOT NULL, UNIQUE	Unique certificate code

8.12 Feedback

Column	Data type	Constraints	Description
Feedback_ID	INT	PK, AUTO_INCREMENT	Unique id of the feedback
Registration_ID	INT	NOT NULL, UNIQUE, FK → Registration(Registration_ID)	Registration giving the feedback
Rating	TINYINT	NOT NULL, CHECK (Rating between 1 and 5)	Rating from 1 to 5
Comments	VARCHAR(255)		Free-text comments
Submitted_At	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of submission

9. SQL Commands Used (DDL, DML) with Sample Outputs

All commands are in the file `event_db.sql` submitted with this report. Outputs below were captured by running the script on a MySQL-compatible server (MariaDB 10.11), using the `mysql` command-line client.

9.1 DDL – Database and Table Creation

```

CREATE DATABASE event_db;
USE event_db;

CREATE TABLE Category (
    Category_ID INT AUTO_INCREMENT PRIMARY KEY,
    Category_Name VARCHAR(60) NOT NULL UNIQUE,
    Description VARCHAR(255)
);

CREATE TABLE Coordinator (
    Coordinator_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(80) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(15),
    Department VARCHAR(60)
);

CREATE TABLE Venue (
    Venue_ID INT AUTO_INCREMENT PRIMARY KEY,
    Venue_Name VARCHAR(120) NOT NULL,
    Location VARCHAR(200),
    Capacity INT NOT NULL,
    CONSTRAINT chk_venue_capacity CHECK (Capacity >= 1)
);

CREATE TABLE Speaker (
    Speaker_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(80) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Expertise VARCHAR(100),
    Bio VARCHAR(255),
    Contact_Phone VARCHAR(15)
);

CREATE TABLE Participant (
    Participant_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(80) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(15),
    Institution VARCHAR(100)
);

CREATE TABLE Event (
    Event_ID INT AUTO_INCREMENT PRIMARY KEY,
    Event_Name VARCHAR(120) NOT NULL,
    Category_ID INT NOT NULL,
    Coordinator_ID INT NOT NULL,
    Event_Date DATE NOT NULL,
    Description VARCHAR(255),
    Status VARCHAR(15) NOT NULL DEFAULT 'planned',
    CONSTRAINT fk_event_category FOREIGN KEY (Category_ID)
        REFERENCES Category(Category_ID),
    CONSTRAINT fk_event_coordinator FOREIGN KEY (Coordinator_ID)
        REFERENCES Coordinator(Coordinator_ID),

```

```

        CONSTRAINT chk_event_status CHECK (Status IN ('planned','open','completed','cancelled'))
    );
CREATE TABLE Session (
    Session_ID INT AUTO_INCREMENT PRIMARY KEY,
    Event_ID   INT NOT NULL,
    Venue_ID   INT NOT NULL,
    Speaker_ID INT,
    Title       VARCHAR(120),
    Start_Time  DATETIME NOT NULL,
    End_Time    DATETIME NOT NULL,
    CONSTRAINT fk_session_event FOREIGN KEY (Event_ID)
        REFERENCES Event(Event_ID) ON DELETE RESTRICT,
    CONSTRAINT fk_session_venue FOREIGN KEY (Venue_ID)
        REFERENCES Venue(Venue_ID),
    CONSTRAINT fk_session_speaker FOREIGN KEY (Speaker_ID)
        REFERENCES Speaker(Speaker_ID),
    CONSTRAINT chk_session_time CHECK (End_Time > Start_Time)
);
CREATE TABLE Registration (
    Registration_ID INT AUTO_INCREMENT PRIMARY KEY,
    Event_ID        INT NOT NULL,
    Participant_ID  INT NOT NULL,
    Registration_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    Status          VARCHAR(15) NOT NULL DEFAULT 'confirmed',
    CONSTRAINT uq_reg_event_participant UNIQUE (Event_ID, Participant_ID),
    CONSTRAINT fk_reg_event FOREIGN KEY (Event_ID)
        REFERENCES Event(Event_ID),
    CONSTRAINT fk_reg_participant FOREIGN KEY (Participant_ID)
        REFERENCES Participant(Participant_ID),
    CONSTRAINT chk_reg_status CHECK (Status IN ('confirmed','cancelled','waitlisted'))
);
CREATE TABLE Payment (
    Payment_ID INT AUTO_INCREMENT PRIMARY KEY,
    Registration_ID INT NOT NULL,
    Amount       DECIMAL(10,2) NOT NULL,
    Mode         VARCHAR(20),
    Payment_Date DATE,
    Status       VARCHAR(15) NOT NULL DEFAULT 'paid',
    CONSTRAINT fk_pay_reg FOREIGN KEY (Registration_ID)
        REFERENCES Registration(Registration_ID),
    CONSTRAINT chk_pay_amount CHECK (Amount >= 0),
    CONSTRAINT chk_pay_status CHECK (Status IN ('paid','pending','refunded'))
);
CREATE TABLE Attendance (
    Attendance_ID INT AUTO_INCREMENT PRIMARY KEY,
    Registration_ID INT NOT NULL,
    Session_ID     INT NOT NULL,
    Marked_At      DATETIME DEFAULT CURRENT_TIMESTAMP,
    Present        TINYINT(1) NOT NULL DEFAULT 1,
    CONSTRAINT uq_att_reg_session UNIQUE (Registration_ID, Session_ID),
    CONSTRAINT fk_att_reg FOREIGN KEY (Registration_ID)
        REFERENCES Registration(Registration_ID),
    CONSTRAINT fk_att_session FOREIGN KEY (Session_ID)
        REFERENCES Session(Session_ID)
);
CREATE TABLE Certificate (
    Certificate_ID INT AUTO_INCREMENT PRIMARY KEY,
    Registration_ID INT NOT NULL UNIQUE,
    Issued_Date    DATE NOT NULL,
    Cert_Code      VARCHAR(30) NOT NULL UNIQUE,
    CONSTRAINT fk_cert_reg FOREIGN KEY (Registration_ID)
        REFERENCES Registration(Registration_ID)
);
CREATE TABLE Feedback (
    Feedback_ID INT AUTO_INCREMENT PRIMARY KEY,
    Registration_ID INT NOT NULL UNIQUE,
    Rating       TINYINT NOT NULL,
    Comments     VARCHAR(255),
    Submitted_At DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_fb_reg FOREIGN KEY (Registration_ID)
        REFERENCES Registration(Registration_ID),
    CONSTRAINT chk_fb_rating CHECK (Rating BETWEEN 1 AND 5)
);

```

```
-- Helpful indexes for the search / join paths used by the application
CREATE INDEX idx_session_event ON Session(Event_ID);
CREATE INDEX idx_session_venue_time ON Session(Venue_ID, Start_Time, End_Time);
CREATE INDEX idx_event_date ON Event(Event_Date);
```

Output: the tables created in the database.

```
mysql> USE event_db;
mysql> SHOW TABLES;
```

```
+-----+
| Tables_in_event_db |
+-----+
| Attendance          |
| Category            |
| Certificate          |
| Coordinator          |
| Event               |
| Feedback            |
| Participant         |
| Payment             |
| Registration         |
| Session             |
| Speaker             |
| V_AVAIL_SEATS        |
| V_CERT_ELIGIBLE     |
| Venue               |
+-----+
```

9.2 Triggers (business rules that need more than a CHECK)

```
DELIMITER //
CREATE TRIGGER trg_session_no_overlap BEFORE INSERT ON Session
FOR EACH ROW
BEGIN
    IF EXISTS (SELECT 1 FROM Session
               WHERE Venue_ID = NEW.Venue_ID
                 AND NEW.Start_Time < End_Time
                 AND NEW.End_Time > Start_Time) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Venue already booked for this time window';
    END IF;
END//

CREATE TRIGGER trg_registration_capacity BEFORE INSERT ON Registration
FOR EACH ROW
BEGIN
    DECLARE cap INT;
    DECLARE taken INT;
    SELECT MIN(V.Capacity) INTO cap
    FROM Session S JOIN Venue V ON V.Venue_ID = S.Venue_ID
    WHERE S.Event_ID = NEW.Event_ID;
    SELECT COUNT(*) INTO taken
    FROM Registration
    WHERE Event_ID = NEW.Event_ID AND Status = 'confirmed';
    IF cap IS NOT NULL AND taken >= cap THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Venue capacity reached for this event';
    END IF;
END//

CREATE TRIGGER trg_certificate_threshold BEFORE INSERT ON Certificate
FOR EACH ROW
BEGIN
    DECLARE total INT;
    DECLARE attended INT;
    SELECT COUNT(*) INTO total
    FROM Session S JOIN Registration R ON R.Event_ID = S.Event_ID
    WHERE R.Registration_ID = NEW.Registration_ID;
    SELECT COUNT(*) INTO attended
    FROM Attendance
    WHERE Registration_ID = NEW.Registration_ID AND Present = 1;
    IF total = 0 OR (attended * 100 / total) < 75 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Attendance below 75% - certificate not allowed';
    END IF;
END//
DELIMITER ;
```

9.3 Views

```
CREATE VIEW V_AVAIL_SEATS AS
SELECT S.Session_ID, V.Venue_Name,
       V.Capacity - (SELECT COUNT(*) FROM Registration R
                     WHERE R.Event_ID = S.Event_ID
                        AND R.Status = 'confirmed') AS Available
FROM Session S
JOIN Venue V ON V.Venue_ID = S.Venue_ID;

CREATE VIEW V_CERT_ELIGIBLE AS
SELECT R.Registration_ID, P.Name AS Participant, E.Event_Name,
       SUM(A.Present) AS Attended,
       (SELECT COUNT(*) FROM Session S
        WHERE S.Event_ID = R.Event_ID) AS Total_Sessions,
       ROUND(SUM(A.Present) * 100 /
             (SELECT COUNT(*) FROM Session S
              WHERE S.Event_ID = R.Event_ID), 1) AS Attendance_Pct
FROM Registration R
JOIN Participant P ON P.Participant_ID = R.Participant_ID
JOIN Event E ON E.Event_ID = R.Event_ID
JOIN Attendance A ON A.Registration_ID = R.Registration_ID
GROUP BY R.Registration_ID, P.Name, E.Event_Name, R.Event_ID
HAVING Attendance_Pct >= 75;
```

9.4 DML – INSERT (sample data)

At least five records were inserted in every table. A few of the INSERT statements are shown here; the complete set is in `event_db.sql`.

```
INSERT INTO Category (Category_Name, Description) VALUES
('Seminar','Expert talks and lectures'),
('Workshop','Hands-on technical sessions'),
('Cultural Fest','Cultural and performing arts events'),
('Hackathon','Timed team coding competitions'),
('Guest Lecture','Industry and alumni interactions');

INSERT INTO Venue (Venue_Name, Location, Capacity) VALUES
('Hall A','Academic Block 1, Floor 1',120),
('Room 3','Academic Block 2, Floor 3',40),
('Auditorium','Main Campus',300),
('Lab 2','Computer Centre, Floor 2',30),
('Board Room','Admin Block, Floor 2',5),
('Seminar Hall','Academic Block 1, Floor 2',80);

INSERT INTO Participant (Name, Email, Phone, Institution) VALUES
('Riya Sharma','riya.sharma@example.com','+91-9876543210','Woxsen University'),
('Vikram Singh','vikram.singh@example.com','+91-9876543211','Woxsen University'),
('Anita Desai','anita.desai@example.com','+91-9876543212','Woxsen University'),
('Rahul Verma','rahul.verma@example.com','+91-9876543213','Woxsen University'),
('Sneha Patel','sneha.patel@example.com','+91-9876543214','Woxsen University');
-- (5 more participants in event_db.sql)

INSERT INTO Registration (Event_ID, Participant_ID) VALUES
(1,1),(1,2),(1,3),(1,6),(1,7), (2,2),(2,4),(2,8), (3,5),(3,6),(3,9),(3,10),
(4,1),(4,3),(4,7), (5,4),(5,8),(5,9), (6,1),(6,2),(6,3),(6,4),(6,5);

INSERT INTO Payment (Registration_ID, Amount, Mode, Payment_Date, Status) VALUES
(1,250.00,'UPI','2026-08-26','paid'),
(2,250.00,'Cash','2026-08-26','paid'),
(5,250.00,'UPI',NULL,'pending');
-- (8 more payments in event_db.sql)
```

Rows inserted per table (query: `SELECT COUNT(*)` on each table):

Table_Name	Rows_Inserted
Category	5
Coordinator	5
Venue	6
Speaker	6
Participant	10
Event	6
Session	8
Registration	23
Payment	11
Attendance	31
Certificate	7
Feedback	7

Sample outputs – SELECT * on key tables:

```
mysql> SELECT Event_ID, Event_Name, Category_ID, Coordinator_ID, Event_Date, Status
FROM Event;
```

Event_ID	Event_Name	Category_ID	Coordinator_ID	Event_Date	Status
1	Algorithms Workshop	2	1	2026-09-04	completed
2	Soft Skills Talk	1	2	2026-09-09	completed
3	Cultural Fest	3	3	2026-09-12	completed
4	Cloud Computing Seminar	1	4	2026-09-16	completed
5	AI Hackathon	4	4	2026-09-19	completed
6	Leadership Roundtable	5	5	2026-09-23	completed

```
mysql> SELECT *
FROM Registration
LIMIT 8;
```

Registration_ID	Event_ID	Participant_ID	Registration_Date	Status
1	1	1	2026-08-25 10:00:00	confirmed
2	1	2	2026-08-25 10:00:00	confirmed
3	1	3	2026-08-25 10:00:00	confirmed
4	1	6	2026-08-25 10:00:00	confirmed
5	1	7	2026-08-25 10:00:00	confirmed
6	2	2	2026-08-25 10:00:00	confirmed
7	2	4	2026-08-25 10:00:00	confirmed
8	2	8	2026-08-25 10:00:00	confirmed

```
mysql> SELECT *
FROM Payment;
```

Payment_ID	Registration_ID	Amount	Mode	Payment_Date	Status
1	1	250.00	UPI	2026-08-26	paid
2	2	250.00	Cash	2026-08-26	paid
3	3	250.00	UPI	2026-08-27	paid
4	4	250.00	Card	2026-08-27	paid
5	5	250.00	UPI	NULL	pending
6	6	0.00	Free	2026-08-30	paid
7	7	0.00	Free	2026-08-30	paid
8	13	150.00	UPI	2026-09-01	paid
9	14	150.00	Card	2026-09-01	paid
10	16	100.00	UPI	2026-09-02	paid
11	17	100.00	Cash	2026-09-02	paid

```
mysql> SELECT *
FROM Attendance
LIMIT 8;
```

Attendance_ID	Registration_ID	Session_ID	Marked_At	Present
1	1	1	2026-10-06 04:41:58	1
2	1	2	2026-10-06 04:41:58	1
3	2	1	2026-10-06 04:41:58	1
4	2	2	2026-10-06 04:41:58	1
5	3	1	2026-10-06 04:41:58	1
6	3	2	2026-10-06 04:41:58	0
7	4	1	2026-10-06 04:41:58	1
8	4	2	2026-10-06 04:41:58	1

9.5 DML – UPDATE and DELETE**UPDATE - mark a pending payment as paid**

```
mysql> UPDATE Payment SET Status = 'paid', Payment_Date = '2026-09-03' WHERE Payment_ID = 5;
```

```
Query OK, 1 row affected (0.001 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

Verify the update

```
mysql> SELECT Payment_ID, Registration_ID, Amount, Status, Payment_Date
FROM Payment WHERE Payment_ID = 5;
```

```
+-----+-----+-----+-----+-----+
| Payment_ID | Registration_ID | Amount | Status | Payment_Date |
+-----+-----+-----+-----+-----+
|          5 |          5 | 250.00 | paid   | 2026-09-03   |
+-----+-----+-----+-----+-----+
```

DELETE - remove one feedback row

```
mysql> DELETE FROM Feedback WHERE Feedback_ID = 7;
```

```
Query OK, 1 row affected (0.001 sec)
```

Verify the delete

```
mysql> SELECT COUNT(*) AS Feedback_Rows FROM Feedback;
```

```
+-----+
| Feedback_Rows |
+-----+
|          6 |
+-----+
```

10. Queries with Outputs

Eleven queries cover joins, aggregate functions (COUNT, SUM, AVG), GROUP BY, HAVING, subqueries, a LEFT JOIN and views.

Query 1: Available seats per event (venue capacity against registrations)

```
SELECT E.Event_Name, V.Venue_Name, V.Capacity,
       COUNT(DISTINCT R.Registration_ID) AS Registered,
       V.Capacity - COUNT(DISTINCT R.Registration_ID) AS Available
FROM Event E
JOIN Session S ON S.Event_ID = E.Event_ID
JOIN Venue V ON V.Venue_ID = S.Venue_ID
LEFT JOIN Registration R ON R.Event_ID = E.Event_ID
GROUP BY E.Event_ID, E.Event_Name, V.Venue_ID, V.Venue_Name, V.Capacity;
```

Output:

```
+-----+-----+-----+-----+-----+
| Event_Name          | Venue_Name | Capacity | Registered | Available |
+-----+-----+-----+-----+-----+
| Algorithms Workshop | Lab 2      | 30       | 5          | 25        |
| Soft Skills Talk    | Room 3     | 40       | 3          | 37        |
| Cultural Fest       | Auditorium | 300      | 4          | 296       |
| Cloud Computing Seminar | Hall A    | 120      | 3          | 117       |
| AI Hackathon        | Seminar Hall | 80       | 3          | 77        |
| Leadership Roundtable | Board Room | 5        | 5          | 0         |
+-----+-----+-----+-----+-----+
```

Query 2: Most registered events (top 5)

```
SELECT E.Event_Name, COUNT(*) AS Total_Registrations
FROM Registration R
JOIN Event E ON E.Event_ID = R.Event_ID
GROUP BY E.Event_ID, E.Event_Name
ORDER BY Total_Registrations DESC
LIMIT 5;
```

Output:

```
+-----+-----+
| Event_Name          | Total_Registrations |
+-----+-----+
| Leadership Roundtable | 5                   |
| Algorithms Workshop  | 5                   |
| Cultural Fest        | 4                   |
| AI Hackathon         | 3                   |
| Cloud Computing Seminar | 3                   |
+-----+-----+
```

Query 3: Attendance percentage per event

```
SELECT E.Event_Name, ROUND(AVG(A.Present) * 100, 1) AS Attendance_Pct
FROM Attendance A
JOIN Session S ON S.Session_ID = A.Session_ID
JOIN Event E ON E.Event_ID = S.Event_ID
GROUP BY E.Event_ID, E.Event_Name;
```

Output:

Event_Name	Attendance_Pct
Algorithms Workshop	80.0
Soft Skills Talk	83.3
Cultural Fest	75.0
Cloud Computing Seminar	100.0
AI Hackathon	66.7
Leadership Roundtable	100.0

Query 4: Participants eligible for certificates (view V_CERT_ELIGIBLE, attendance >= 75%)

```
SELECT * FROM V_CERT_ELIGIBLE;
```

Output:

Registration_ID	Participant	Event_Name	Attended	Total_Sessions	Attendance_Pct
1	Riya Sharma	Algorithms Workshop	2	2	100.0
2	Vikram Singh	Algorithms Workshop	2	2	100.0
4	Aman Gupta	Algorithms Workshop	2	2	100.0
6	Vikram Singh	Soft Skills Talk	2	2	100.0
8	Kabir Khan	Soft Skills Talk	2	2	100.0
9	Sneha Patel	Cultural Fest	1	1	100.0
10	Aman Gupta	Cultural Fest	1	1	100.0
12	Manish Reddy	Cultural Fest	1	1	100.0
13	Riya Sharma	Cloud Computing Seminar	1	1	100.0
14	Anita Desai	Cloud Computing Seminar	1	1	100.0
15	Pooja Nair	Cloud Computing Seminar	1	1	100.0
16	Rahul Verma	AI Hackathon	1	1	100.0
17	Kabir Khan	AI Hackathon	1	1	100.0
19	Riya Sharma	Leadership Roundtable	1	1	100.0
20	Vikram Singh	Leadership Roundtable	1	1	100.0
21	Anita Desai	Leadership Roundtable	1	1	100.0
22	Rahul Verma	Leadership Roundtable	1	1	100.0
23	Sneha Patel	Leadership Roundtable	1	1	100.0

Query 5: Total revenue per event

```
SELECT E.Event_Name, SUM(P.Amount) AS Revenue
FROM Payment P
JOIN Registration R ON R.Registration_ID = P.Registration_ID
JOIN Event E ON E.Event_ID = R.Event_ID
WHERE P.Status = 'paid'
GROUP BY E.Event_ID, E.Event_Name
ORDER BY Revenue DESC;
```

Output:

Event_Name	Revenue
Algorithms Workshop	1000.00
Cloud Computing Seminar	300.00
AI Hackathon	200.00
Soft Skills Talk	0.00

Query 6: Speaker schedule

```
SELECT Sp.Name AS Speaker, E.Event_Name, V.Venue_Name, S.Start_Time, S.End_Time
FROM Session S
JOIN Speaker Sp ON Sp.Speaker_ID = S.Speaker_ID
JOIN Event E ON E.Event_ID = S.Event_ID
JOIN Venue V ON V.Venue_ID = S.Venue_ID
ORDER BY S.Start_Time;
```

Output:

Speaker	Event_Name	Venue_Name	Start_Time	End_Time
Dr. Suresh Babu	Algorithms Workshop	Lab 2	2026-09-04 10:00:00	2026-09-04 12:00:00
Dr. Suresh Babu	Algorithms Workshop	Lab 2	2026-09-04 14:00:00	2026-09-04 16:00:00
Neha Kapoor	Soft Skills Talk	Room 3	2026-09-09 11:00:00	2026-09-09 12:30:00
Neha Kapoor	Soft Skills Talk	Room 3	2026-09-09 12:30:00	2026-09-09 13:30:00
Rohit Menon	Cultural Fest	Auditorium	2026-09-12 09:00:00	2026-09-12 13:00:00
Divya Iyer	Cloud Computing Seminar	Hall A	2026-09-16 10:00:00	2026-09-16 12:00:00
Arjun Das	AI Hackathon	Seminar Hall	2026-09-19 09:00:00	2026-09-19 17:00:00
Sana Qureshi	Leadership Roundtable	Board Room	2026-09-23 15:00:00	2026-09-23 16:30:00

Query 7: Feedback summary: events rated above the overall average (subquery + HAVING)

```
SELECT E.Event_Name, ROUND(AVG(F.Rating), 2) AS Avg_Rating, COUNT(F.Comments) AS Comments
FROM Feedback F
JOIN Registration R ON R.Registration_ID = F.Registration_ID
JOIN Event E ON E.Event_ID = R.Event_ID
GROUP BY E.Event_ID, E.Event_Name
HAVING AVG(F.Rating) > (SELECT AVG(Rating) FROM Feedback);
```

Output:

Event_Name	Avg_Rating	Comments
Algorithms Workshop	4.67	3
Cultural Fest	5.00	1

Query 8: Pending payments list

```
SELECT R.Registration_ID, P.Name, P.Phone, E.Event_Name, Y.Amount
FROM Payment Y
JOIN Registration R ON R.Registration_ID = Y.Registration_ID
JOIN Participant P ON P.Participant_ID = R.Participant_ID
JOIN Event E ON E.Event_ID = R.Event_ID
WHERE Y.Status = 'pending';
```

Output:

Registration_ID	Name	Phone	Event_Name	Amount
5	Pooja Nair	+91-9876543216	Algorithms Workshop	250.00

Query 9: Returning participants (registered for more than one event)

```
SELECT P.Name, P.Email, COUNT(*) AS Events_Registered
FROM Participant P
JOIN Registration R ON R.Participant_ID = P.Participant_ID
WHERE R.Status = 'confirmed'
GROUP BY P.Participant_ID, P.Name, P.Email
HAVING COUNT(*) > 1
ORDER BY Events_Registered DESC;
```

Output:

Name	Email	Events_Registered
Rahul Verma	rahul.verma@example.com	3
Anita Desai	anita.desai@example.com	3
Vikram Singh	vikram.singh@example.com	3
Riya Sharma	riya.sharma@example.com	3
Pooja Nair	pooja.nair@example.com	2
Aman Gupta	aman.gupta@example.com	2
Sneha Patel	sneha.patel@example.com	2
Ishita Rao	ishita.rao@example.com	2
Kabir Khan	kabir.khan@example.com	2

Query 10: Sessions by venue and day (operations view)

```
SELECT V.Venue_Name, DATE(S.Start_Time) AS Day,
       TIME(S.Start_Time) AS Starts, TIME(S.End_Time) AS Ends, E.Event_Name
FROM Session S
JOIN Venue V ON V.Venue_ID = S.Venue_ID
JOIN Event E ON E.Event_ID = S.Event_ID
ORDER BY V.Venue_Name, S.Start_Time;
```

Output:

Venue_Name	Day	Starts	Ends	Event_Name
Auditorium	2026-09-12	09:00:00	13:00:00	Cultural Fest
Board Room	2026-09-23	15:00:00	16:30:00	Leadership Roundtable
Hall A	2026-09-16	10:00:00	12:00:00	Cloud Computing Seminar
Lab 2	2026-09-04	10:00:00	12:00:00	Algorithms Workshop
Lab 2	2026-09-04	14:00:00	16:00:00	Algorithms Workshop
Room 3	2026-09-09	11:00:00	12:30:00	Soft Skills Talk
Room 3	2026-09-09	12:30:00	13:30:00	Soft Skills Talk
Seminar Hall	2026-09-19	09:00:00	17:00:00	AI Hackathon

Query 11: Available seats per session (view V_AVAIL_SEATS)

```
SELECT * FROM V_AVAIL_SEATS;
```

Output:

Session_ID	Venue_Name	Available
1	Lab 2	25
2	Lab 2	25
3	Room 3	37
4	Room 3	37
5	Auditorium	296
6	Hall A	117
7	Seminar Hall	77
8	Board Room	0

11. UI Design and Screenshots

11.1 Design Approach

- The user interface is a web-based prototype named **EVENTRA – Venue OS** that runs in the browser.
- It has three role-based views: **Administrator**, **Organizer** (the Coordinator role) and **Participant**. The sidebar menu changes according to the logged-in role.

- Common layout on every screen: left sidebar for navigation, summary cards at the top, lists or tables below and quick actions on the side.
- The UI only collects input and shows results. Validation and storage are done by the application logic and the database.
- Every insert, delete or update is shown on screen straight away and can be cross-checked in MySQL.
- Extra features in the prototype: in-app notifications, QR ticket for check-in and a waitlist when capacity is full.

11.2 Screens

Screen	Role	What it shows	Main database tables
Admin dashboard	Administrator	System-wide counters (users, events, venues, registrations, venue slots), upcoming events, events by category	Event, Venue, Registration, Category
Organizer dashboard	Organizer	My events, seat fill %, tickets, attendance rate, event roster and quick actions	Event, Registration, Attendance
Create event	Administrator / Organizer	Form: name, description, category, type, status, capacity, date, start / end time, venue	Event, Session, Venue (overlap check)
Venues	Administrator	Venue cards with seats, location, amenities, bookings, rate; Schedule / Edit / Maintenance / Delete	Venue, Session
Notifications	All roles	Totals, unread count, event scheduled and new registration messages; Mark read	Registration, Session
Participant dashboard	Participant	My tickets, upcoming events, attended count, upcoming registrations, QR ticket	Registration, Event, Attendance
Reports	Administrator / Organizer	Registrations, fill rate, attendance %, venue hours; registrations per month; venue utilization; attendance by event	Views and GROUP BY queries

11.3 Prototype Screenshots

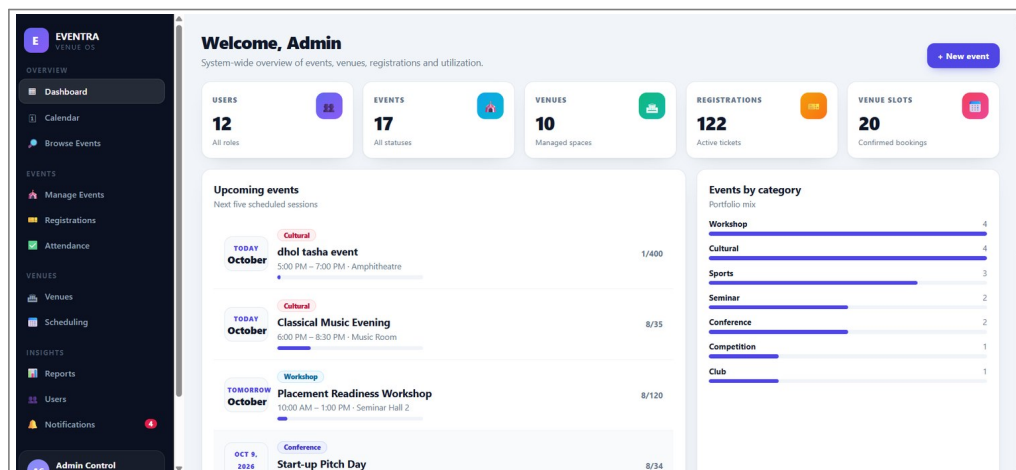


Figure 11.1 – Administrator dashboard. Shows 12 users, 17 events, 10 venues, 122 registrations and 20 confirmed venue slots, with upcoming events and an events-by-category chart (COUNT / GROUP BY queries).

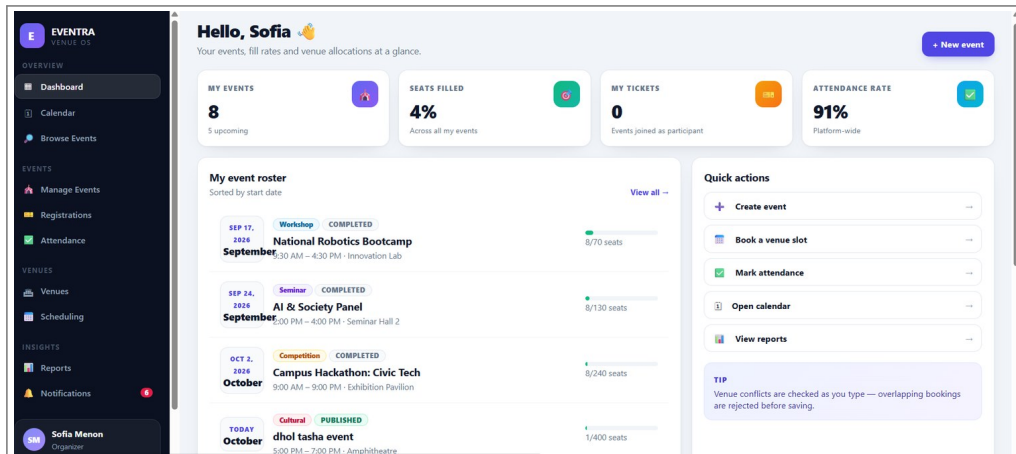


Figure 11.2 – Organizer dashboard. Organizer view: my events, seat-fill percentage, attendance rate and a roster of own events with seats filled, plus quick actions to create events and mark attendance.

Figure 11.3 – Create event form (insert operation). Insert form for a new event: details, category, type, status, capacity (extra registrations join the waitlist), date, start and end time, and venue. Venue overlaps are checked before saving.

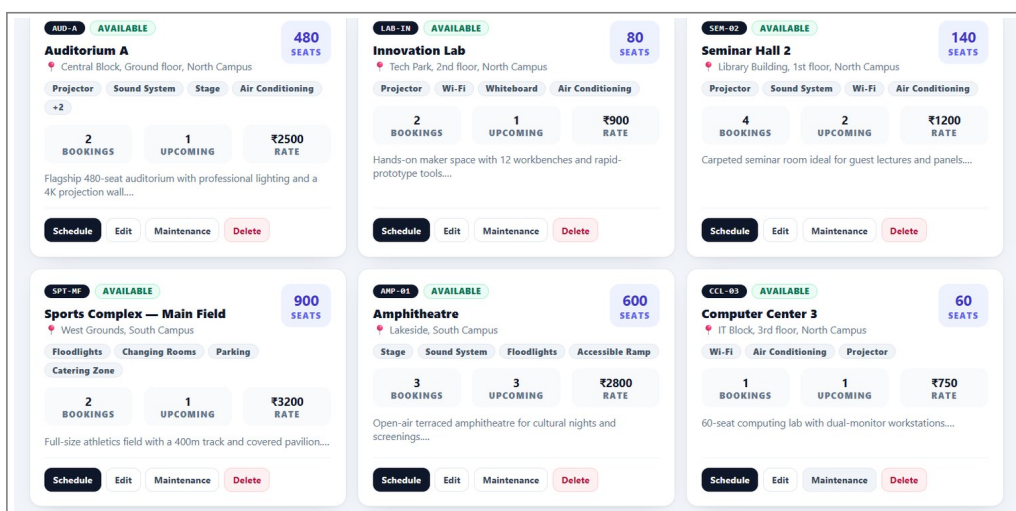


Figure 11.4 – Venues screen. Venue cards with seats, location, amenities, bookings and rate. Buttons: Schedule, Edit, Maintenance and Delete (update and delete operations on Venue).

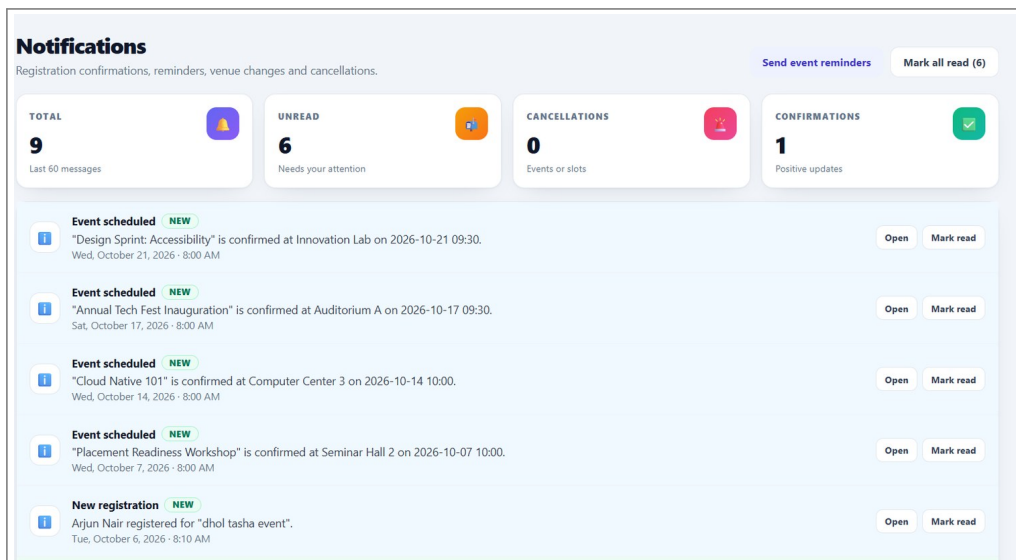


Figure 11.5 – Notifications screen. Total, unread, cancellation and confirmation counts, with messages such as "Event scheduled" and "New registration" and a Mark read action.

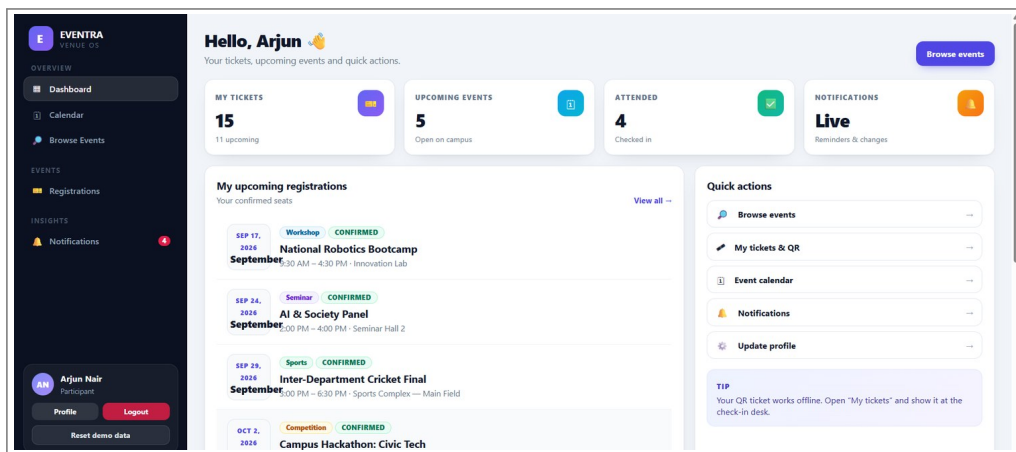


Figure 11.6 – Participant dashboard. Participant view: my tickets, upcoming events, attended events and confirmed registrations, with quick actions including the QR ticket for check-in.

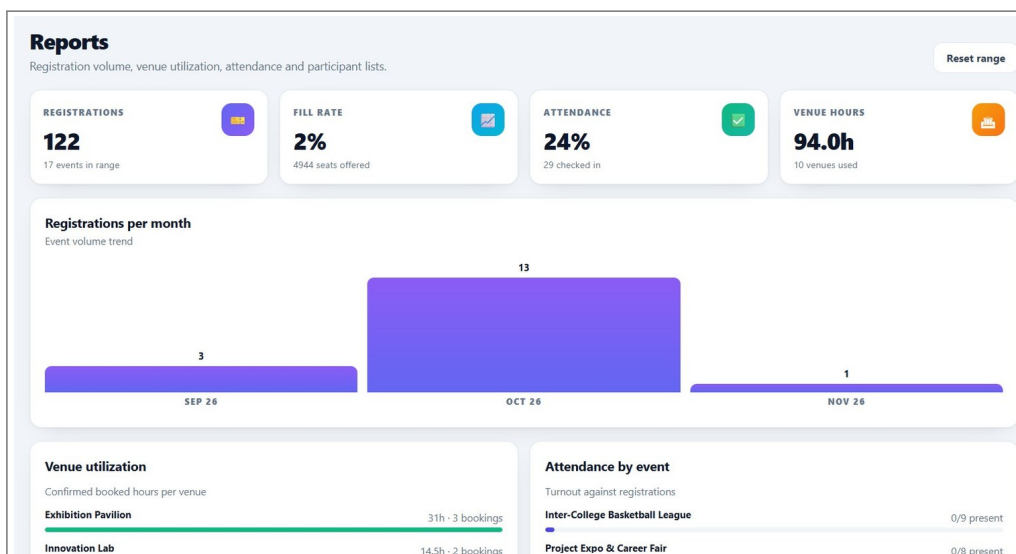


Figure 11.7 – Reports screen. Registrations, fill rate, attendance and venue hours; registrations per month; venue utilization and attendance by event, produced with aggregate queries and views.

12. Implementation Details

12.1 Technology Stack

Layer	Technology	Responsibility
Presentation	Next.js 16, React 19, Tailwind CSS 4 (EVENTRA)	Role-based dashboards, forms, tables, notifications, QR tickets
Application	Next.js server-side code in TypeScript	Validation, CRUD, transactions, report queries
Data access	Drizzle ORM with node-postgres (pg)	Typed schema, parameterised queries, transactions
Database	PostgreSQL (prototype); MySQL 8.0 script event_db.sql	Tables, constraints, indexes, views – the authoritative store

12.2 Three-Layer Architecture

- **Layer 1 – Presentation:** the user touches only this layer; the schema stays hidden.
- **Layer 2 – Application:** business rules are checked once here for fast feedback.
- **Layer 3 – Database:** the same rules are enforced again by constraints as the final guard.

12.3 Registration Flow

1. User submits the registration form (event, participant, payment).
2. Application pre-checks: duplicate registration, payment ≥ 0 .
3. A transaction is opened; Registration is inserted, then Payment, followed by a single COMMIT.
4. The database checks FK, UNIQUE and CHECK constraints. Any failure causes a ROLLBACK.
5. The UI shows a success message, or the error text, and refreshes the list.

12.4 Key Design Decisions

- All queries go through Drizzle ORM, which sends parameterised statements and avoids SQL injection.
- Foreign keys use RESTRICT (not CASCADE) in the SQL design so that registration, payment and attendance history is never lost silently.
- Indexes on Session(Event_ID), Session(Venue_ID, Start_Time, End_Time) and Event(Event_Date) speed up joins and overlap checks.
- Rules are enforced twice: in the application for quick feedback and in the database as the final authority.
- Connection details are kept in environment variables (dotenv), not in the source code.

12.5 Database Connectivity

```
// db.ts
import "dotenv/config";
import { drizzle } from "drizzle-orm/node-postgres";
import { Pool } from "pg";

const pool = new Pool({ connectionString: process.env.DATABASE_URL });
export const db = drizzle(pool);
```

12.6 Key Code Snippets

(a) Register a participant – transaction with rollback

```
export async function registerParticipant(
  eventId: number, participantId: number, amount: number, mode: string) {
  if (amount < 0) throw new Error("Payment amount cannot be negative");

  const existing = await db.select().from(registration).where(
    and(eq(registration.eventId, eventId),
      eq(registration.participantId, participantId)));
  if (existing.length > 0)
    throw new Error("Participant is already registered for this event");

  return db.transaction(async (tx) => {          // BEGIN
    const [reg] = await tx.insert(registration)
      .values({ eventId, participantId })
      .returning({ id: registration.registrationId });
    await tx.insert(payment).values({
      registrationId: reg.id, amount: String(amount), mode, status: "paid" });
    return reg.id;          // COMMIT; any thrown error causes ROLLBACK
  });
}
```

(b) View registrations – join query for the table

```
export async function listRegistrations() {
  return db.select({
    id: registration.registrationId,
    participant: participant.name,
    event: event.eventName,
    status: registration.status })
    .from(registration)
    .innerJoin(participant, eq(participant.participantId, registration.participantId))
    .innerJoin(event, eq(event.eventId, registration.eventId))
    .orderBy(registration.registrationId);
}
```

(c) Delete a registration with its dependent rows

```
export async function deleteRegistration(regId: number) {
  await db.transaction(async (tx) => {
    await tx.delete(feedback).where(eq(feedback.registrationId, regId));
    await tx.delete(certificate).where(eq(certificate.registrationId, regId));
    await tx.delete(attendance).where(eq(attendance.registrationId, regId));
    await tx.delete(payment).where(eq(payment.registrationId, regId));
    await tx.delete(registration).where(eq(registration.registrationId, regId));
  });
}
```

13. Testing

Approach: for every business rule there is one failing case and one passing case. Constraint violations are tried directly through SQL, so a bad row must be refused with a clear message and never silently overwritten. Results below are the actual outputs from the server.

ID	Test case	Input / condition	Rule	Expected	Actual result	Status
TC01	Duplicate registration	Same participant, same event, twice	Rule 3	Rejected	ERROR 1062 (23000): Duplicate entry '1-1' for key 'uq_reg_event_participant'	Pass
TC02	Venue capacity exceeded	Event 6 (Board Room, capacity 5) already has 5 registrations	Rule 2	Rejected	ERROR 1644 (45000): Venue capacity reached for this event	Pass
TC03	Venue time-window overlap	New session in Lab 2, 11:00-13:00, overlaps 10:00-12:00	Rule 1	Prevented	ERROR 1644 (45000): Venue already booked for this time window	Pass
TC04	Negative payment amount	Amount = -50	CHECK	Rejected	ERROR 4025 (23000): CONSTRAINT chk_pay_amount failed for event_db.Payment	Pass
TC05	Invalid feedback rating	Rating = 6	CHECK	Rejected	ERROR 4025 (23000): CONSTRAINT chk_fb_rating failed for event_db.Feedback	Pass
TC06	Registration for a missing participant	Participant_ID = 99 does not exist	FK	Rejected	ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (event_db.Registration, CONSTRAI...	Pass
TC07	Certificate below threshold	Registration 3 attended 1 of 2 sessions (50%)	Rule 4	Rejected	ERROR 1644 (45000): Attendance below 75% - certificate not allowed	Pass
TC08	Delete an event that has sessions	Event 1 has sessions and registrations	FK RES TRICT	Rejected	ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (event_db.Registration, CONS...	Pass
TC09	Valid registration	New participant for event, free seat	Happy path	Accepted	Query OK, 1 row affected	Pass
TC10	Certificate availability	Registration 10 attended 1 of 1 session (100%)	Rule 4	Accepted	Query OK, 1 row affected	Pass

SQL used for each test case

```
-- TC01: Duplicate registration
INSERT INTO Registration (Event_ID, Participant_ID)
VALUES (1, 1);

-- TC02: Venue capacity exceeded
INSERT INTO Registration (Event_ID, Participant_ID)
VALUES (6, 6);

-- TC03: Venue time-window overlap
INSERT INTO Session (Event_ID, Venue_ID, Speaker_ID, Title,
Start_Time, End_Time)
VALUES (5, 4, 5, 'Extra Lab', '2026-09-04 11:00:00', '2026-09-04 13:00:00');

-- TC04: Negative payment amount
INSERT INTO Payment (Registration_ID, Amount, Mode, Status)
VALUES (9, -50, 'UPI', 'paid');

-- TC05: Invalid feedback rating
INSERT INTO Feedback (Registration_ID, Rating)
VALUES (10, 6);

-- TC06: Registration for a missing participant
INSERT INTO Registration (Event_ID, Participant_ID)
VALUES (1, 99);

-- TC07: Certificate below threshold
INSERT INTO Certificate (Registration_ID, Issued_Date, Cert_Code)
VALUES (3, '2026-09-06', 'CERT-ALG-0003');

-- TC08: Delete an event that has sessions
DELETE FROM Event WHERE Event_ID = 1;

-- TC09: Valid registration
INSERT INTO Registration (Event_ID, Participant_ID)
VALUES (4, 2);

-- TC10: Certificate availability
INSERT INTO Certificate (Registration_ID, Issued_Date, Cert_Code)
VALUES (10, '2026-09-14', 'CERT-CUL-0010');
```

Summary

- 10 of 10 test cases passed.
- Duplicate registration, capacity overflow, venue overlap, negative payment, invalid rating, missing parent row, early certificate and deleting a parent row were all blocked by the database.
- Valid registration and an eligible certificate were accepted.

14. Conclusion and Future Enhancements

14.1 Conclusion

- A centralized, validated database was designed for event registration and venue scheduling with 12 entities in 3NF.
- Business rules (venue overlap, capacity, duplicates, certificate threshold, non-negative payment) are enforced by the database itself.
- Joins, GROUP BY, HAVING, subqueries and views give ready reports on seats, attendance, revenue, feedback and schedules.
- The EVENTRA web prototype works as a thin layer over the database; every change made in the UI is reflected in the data.
- The project gave practical experience in ER modeling, normalization, constraints, triggers, transactions and SQL.

14.2 Future Enhancements

1. Online payment gateway for real fee collection.
2. Email and SMS delivery of the notifications that the prototype currently shows inside the app.
3. QR-code based check-in at the door (the prototype already issues a QR ticket).
4. Mobile application for participants.
5. Automated PDF certificate generation.
6. Analytics dashboard with charts over the existing views.
7. Cloud deployment of the database and application.
8. Role-based login with separate database users for administrator, coordinator and participant.

15. References

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
- [2] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
- [3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill, 2003.
- [4] Oracle Corporation, *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
- [5] Oracle Corporation, *MySQL Connector/Python Developer Guide*. <https://dev.mysql.com/doc/connector-python/en/>
- [6] Python Software Foundation, *tkinter – Python interface to Tcl/Tk*. <https://docs.python.org/3/library/tkinter.html>
- [7] DBMS course lecture notes and project guidelines, Woxsen University, 2026.

16. Contribution of Each Member

Member	Roll number	Contribution
Devesh Wattamwar	25WU0101030	Requirement analysis; ER diagram; relational schema and normalization; table creation (DDL) with constraints and triggers
Sarthak Borse	25WU0101025	Sample data (DML) with 5+ records per table; SQL queries, views and reports; test cases and results
Dhruv Porwal	25WU0101031	EVENTRA UI prototype and database connectivity; insert, view and delete screens; screenshots; report and presentation

All three members took part in design discussions, reviewed each other's work and prepared for the final presentation together.

17. Appendix: GitHub Repository Link

Repository: <https://github.com/porwaldhruv2007/Eventra>

Eventra is an event registration and venue scheduling web application (Venue OS) that manages events, venue slot bookings, ticket registrations, user roles and attendance tracking, with reports on space utilization and event capacity.

File in the repository	Purpose
package.json	Project name, scripts and dependencies (Next.js, React, Drizzle ORM, pg, qrcode, Tailwind CSS)
drizzle.config.json	Drizzle ORM / drizzle-kit configuration (schema and migrations)
next.config.ts	Next.js configuration
tsconfig.json	TypeScript compiler options
postcss.config.mjs	PostCSS / Tailwind CSS configuration
eslint.config.mjs	ESLint rules for code quality

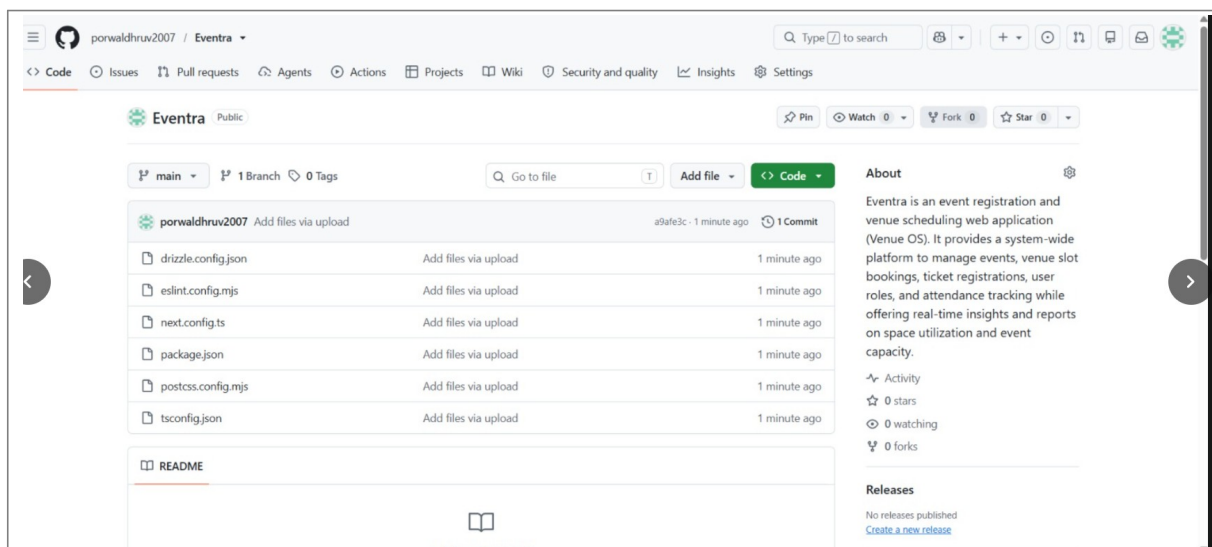


Figure 17.1 – Project files uploaded to the GitHub repository (porwaldhruv2007/Eventra)